BÀI TẬP LỚN – CẤU TRÚC RỜI RẠC

A. Thông tin nhóm

Nhóm: 03

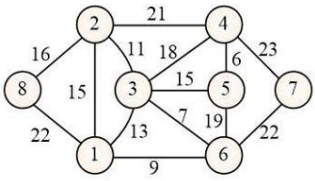
Trưởng nhóm: Nguyễn Võ Quang Huy

Bảng phân công công việc và mức độ đóng góp:

TT	Họ tên	Công việc	Tỷ lệ đóng góp
1	Lê Hoàng Huấn	Code bài 2	15%
2	Nguyễn Hoàng Gia Hưng	Code bài 3	15%
3	Trần Lê Phước Gia Hưng	Giải tay bài 2	15%
4	Đặng Lê Minh Huy	Giải tay bài 3	15%
5	Mai Quốc Huy	Code bài 1	15%
6	Nguyễn Võ Quang Huy	Hỗ trợ các bạn giải bài tập bằng tay, làm file word, kiểm tra, fix lỗi bài làm (code)	25%
Tổng			**100%**

B. Trình bày kết quả 3 bài tập

<u>Câu 1</u>: Viết chương trình đọc vào một đơn đồ thị vô hướng ở dạng ma trận kề, sau đó xuất ra dãy bậc của tất cả các đỉnh của nó. Áp dụng cho đồ thị sau, chụp hình kết quả trình bày vào file word.



1.1. Mục tiêu

- Yêu cầu: Viết chương trình nhập một đơn đồ thị vô hướng, xuất ra dạng ma trận kề và bậc của các đỉnh.

1.2. Cơ sở lý thuyết

- Ma trận kề của đơn đồ thị G = (V, E) có n đỉnh là ma trận vuông cấp n, trong đó:

• a_{ij} = 1 nếu (i, j) thuộc E

• a_{ij} = 0 nếu (i, j) không thuộc E

- Bậc của đỉnh i, kí hiệu là deg(i) bằng tổng số cung (cạnh) kề với đỉnh i, trong đó mỗi khuyên được tính hai lần.

- Đỉnh treo nếu deg(i) = 1, đỉnh cô lập nếu deg(i) = 0.

1.3. Thuật toán, Cấu trúc dữ liệu, Ưu và nhược điểm

- Thuật toán: Sử dụng ngôn ngữ C++

- Sử dụng 2 vòng lặp lồng nhau. Khi nhập giá trị tại hàng i, nếu là 1 thì tăng biến đếm deg[i] thêm 1.
- Xuất ma trận dưới dạng bảng căn biên (dùng setw) và in danh sách bậc đã tính.

- Cấu trúc dữ liệu:

- Ma trận kề (vector<vector<int>>): Dùng mảng 2 chiều kích thước n x n để lưu trữ các cạnh 1 là có cạnh, 0 là không có cạnh.
- Mảng lưu bậc (vector<int>): Mảng 1 chiều để tích lũy số lượng cạnh kết nối với mỗi đỉnh.

- Ưu và nhược điểm:

- Ưu: Đơn giản, dễ cài đặt, kiểm tra nhanh giữa 2 đỉnh có cạnh hay không.
- Nhược: Tốn bộ nhớ n^2, không tối ưu cho đồ thị có hàng nghìn đỉnh nhưng ít cạnh.

1.4. Ma trận kề của đồ thị

	1	2	3	4	5	6	7	8
1	0	1	1	0	0	1	0	1
2	1	0	1	1	0	0	0	1
3	1	1	0	1	1	1	0	0
4	0	1	1	0	1	0	1	0
5	0	0	1	1	0	1	0	0
6	1	0	1	0	1	0	1	0
7	0	0	0	1	0	1	0	0
8	1	1	0	0	0	0	0	0

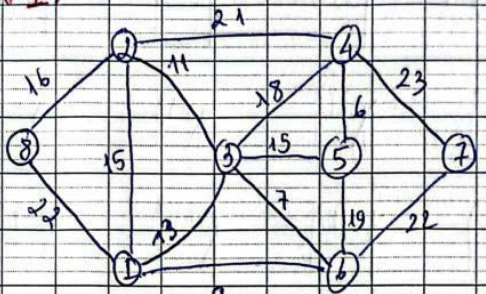
1.5. Bậc của các đỉnh

Đỉnh	Bậc
1	4
2	4
3	5
4	4

Đỉnh	Bậc
5	3
6	4
7	2
8	2

1.6. Giải bài toán bằng tay

Bài 1:



g

- Bậc của đỉnh i bằng tổng số cung (cạnh) kề với đỉnh i, khuyên tại i được tính là 2 lần

Ma trận kề:

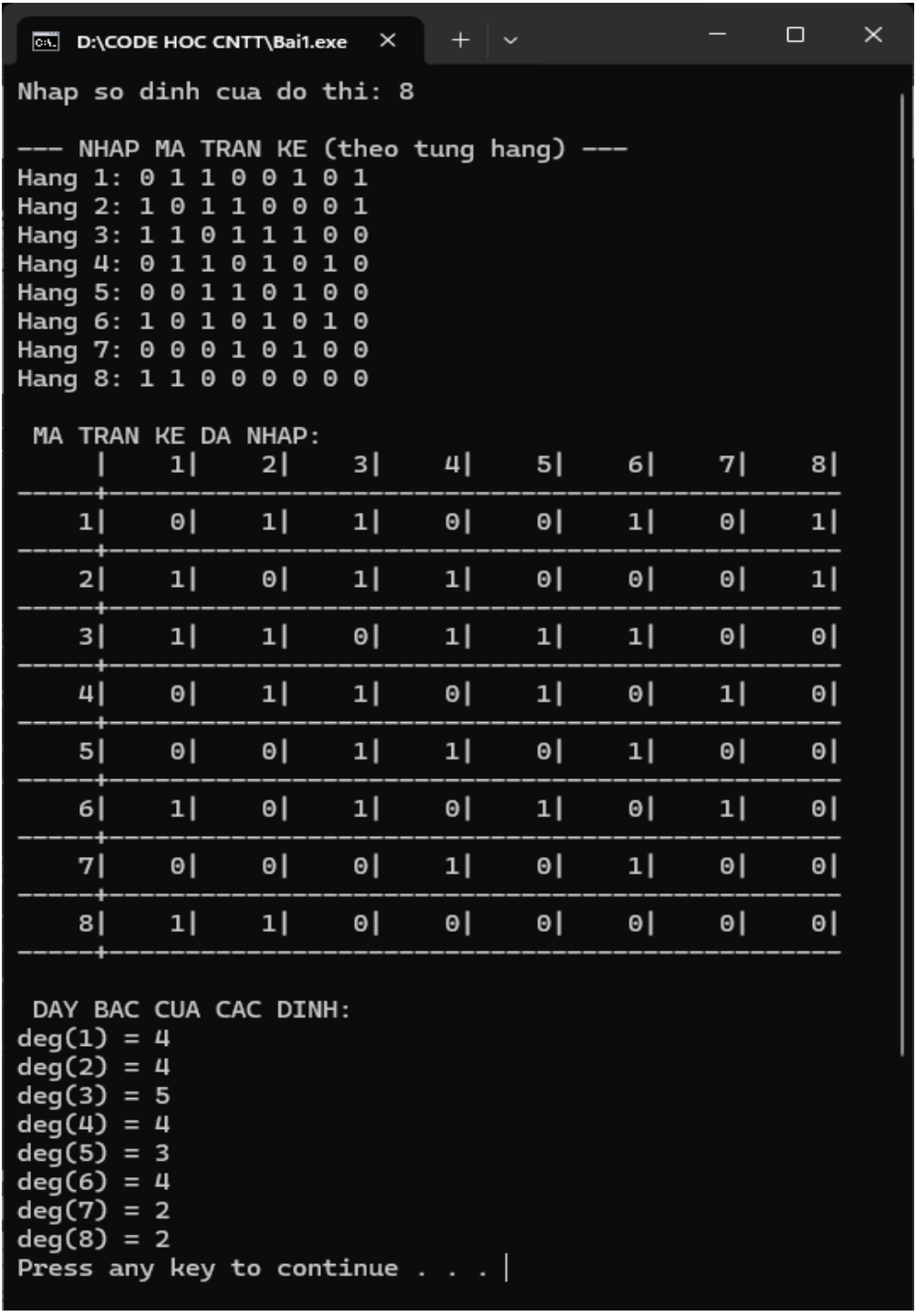
	1	2	3	4	5	6	7	8	
1	0	1	1	0	0	1	0	1	deg(1) = 4
2	1	0	1	1	0	0	0	1	deg(2) = 4
3	1	1	0	1	1	1	0	0	deg(3) = 5
4	0	1	1	0	1	0	1	0	deg(4) = 4
5	0	0	1	1	0	1	0	0	deg(5) = 3
6	1	0	1	0	1	0	1	0	deg(6) = 4
7	0	0	0	1	0	1	0	0	deg(7) = 2
8	1	1	0	0	0	0	0	0	deg(8) = 2

- Tổng các phần tử trên 1 dòng (1 cột) của ma trận bằng bậc của đỉnh tương ứng

TTBOOK

CS Được quét bằng CamScanner

1.7. Giải bài toán bằng code



1.8. Kết luận

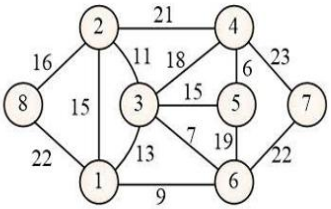
Chương trình đã thực hiện được các yêu cầu sau:

• Đọc vào đồ thị vô hướng có trọng số với n đỉnh và m cạnh.

• Lưu trữ đồ thị dưới dạng ma trận kề A[n][n].

• Xuất ra ma trận kề và bậc của từng đỉnh.

<u>Câu 2</u>: Viết chương trình đọc vào một đơn đồ thị vô hướng ở dạng ma trận kề, sau đó nhập vào một đỉnh và xuất ra dãy đỉnh được duyệt theo thuật toán DFS. Áp dụng cho đồ thị dưới đây, chạy code bằng computer và bằng tay, chụp hình các kết quả trình bày vào file word.



2.1. Mục tiêu

- Viết chương trình nhập vào một đơn đồ thị vô hướng dưới dạng ma trận kề, áp dụng thuật toán DFS vào đồ thị, sau đó xuất ra ma trận kề, nhập đỉnh bắt đầu duyệt và thứ tự duyệt từ đỉnh bắt đầu.

2.2. Cơ sở lý thuyết

- Định nghĩa: DFS (Depth First Search) là thuật toán duyệt hoặc tìm kiếm trên đồ thị bằng cách đi xa nhất có thể dọc theo mỗi nhánh trước khi quay lui.

- Nguyên lý: Sử dụng cấu trúc dữ liệu Ngăn xếp (Stack) hoặc phương pháp Đệ quy.

- Cơ chế: Đánh dấu các đỉnh đã thăm để tránh việc lặp vô hạn trong đồ thị có chu trình.

- Các bước thực hiện:

  - Đánh dấu đỉnh xuất phát là đã thăm, in ra đỉnh đó.
  - Với mỗi đỉnh kề chưa thăm (duyệt theo thứ tự tăng dần), gọi đệ quy DFS.
  - Tiếp tục cho đến khi tất cả đỉnh liên thông được thăm.

2.3. Thuật toán, Cấu trúc dữ liệu, Ưu và nhược điểm

- Thuật toán:

- Cơ chế: Ưu tiên đi sâu nhất có thể vào một nhánh trước khi quay lui để thử nhánh khác.
- Các bước: Thăm đỉnh → Đánh dấu → Gọi đệ quy cho láng giềng chưa thăm.

- Cấu trúc dữ liệu:

- Ma trận kề (vector<vector<int>> A): Lưu trữ các kết nối giữa các đỉnh. A[u][v] = 1 nghĩa là có cạnh nối từ u đến v.
- Mảng đánh dấu (vector<bool> visited): Lưu trạng thái đã thăm của từng đỉnh (tránh lặp vô hạn và đảm bảo mỗi đỉnh chỉ thăm 1 lần).

- Ưu và nhược điểm:

- Ưu: Cài đặt rất gọn, hữu ích để kiểm tra tính liên thông hoặc tìm thành phần liên thông.
- Nhược:
  - Ma trận kề: Tốn bộ nhớ và chạy chậm khi đồ thị ít cạnh.
  - Đệ quy: Có thể gây tràn ngăn xếp (Stack Overflow) nếu đồ thị quá lớn.
  - Đường đi: Không đảm bảo tìm được đường đi ngắn nhất như BFS.

2.4. Thiết kế chương trình

- Ngôn ngữ: Sử dụng ngôn ngữ C++ để hoàn thành bài toán

- Sử dụng mảng 2 chiều và phương pháp đệ quy để xử lý bài toán

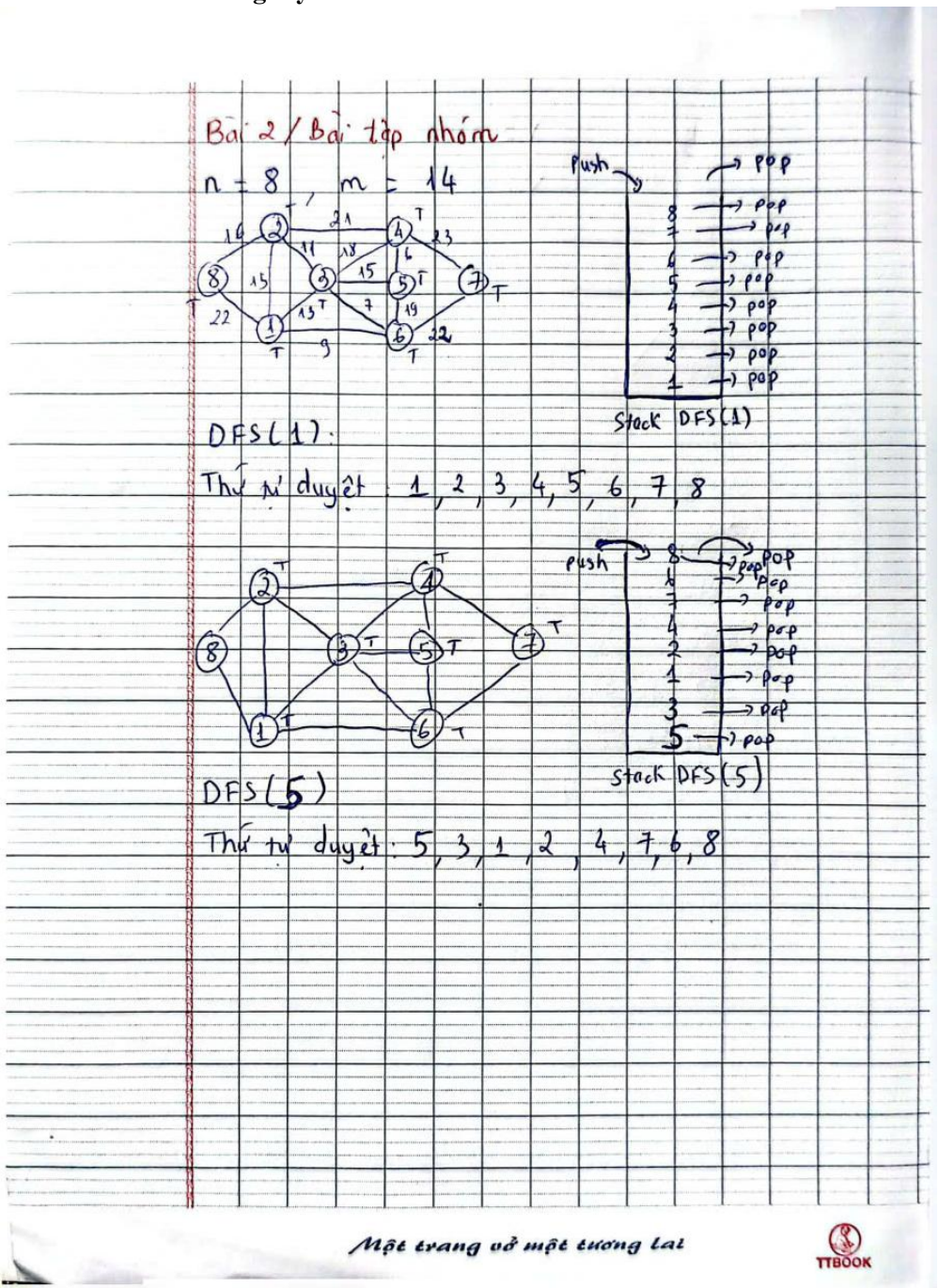
- Lưu đồ thuật toán:

Nhập dữ liệu ma trận kề của đồ thị → Chọn đỉnh bắt đầu duyệt → Tính toán (gọi hàm đệ quy DFS để xử lý bài toán) → Xuất kết quả

2.5. Ma trận kề của đồ thị

	1	2	3	4	5	6	7	8
1	**0**	**1**	**1**	**0**	**0**	**1**	**0**	**1**
2	**1**	**0**	**1**	**1**	**0**	**0**	**0**	**1**
3	**1**	**1**	**0**	**1**	**1**	**1**	**0**	**0**
4	**0**	**1**	**1**	**0**	**1**	**0**	**1**	**0**
5	**0**	**0**	**1**	**1**	**0**	**1**	**0**	**0**
6	**1**	**0**	**1**	**0**	**1**	**0**	**1**	**0**
7	**0**	**0**	**0**	**1**	**0**	**1**	**0**	**0**
8	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**

2.6. Giải bài toán bằng tay



Một trang vở một tương lai
TTBOOK
Được quét bằng CamScanner

2.7. Giải bài toán bằng code

```
D:\CAU TRUC ROI RAC\Bai2.e:

Nhap so luong dinh cua do thi: 8

--- NHAP MA TRAN KE (0 hoac 1) ---
Hang 1 (nhap 8 so): 0 1 1 0 0 1 0 1
Hang 2 (nhap 8 so): 1 0 1 1 0 0 0 1
Hang 3 (nhap 8 so): 1 1 0 1 1 1 0 0
Hang 4 (nhap 8 so): 0 1 1 0 1 0 1 0
Hang 5 (nhap 8 so): 0 0 1 1 0 1 0 0
Hang 6 (nhap 8 so): 1 0 1 0 1 0 1 0
Hang 7 (nhap 8 so): 0 0 0 1 0 1 0 0
Hang 8 (nhap 8 so): 1 1 0 0 0 0 0 0

 MA TRAN KE DA NHAP:
    |   1|   2|   3|   4|   5|   6|   7|   8|
----+----+----+----+----+----+----+----+----+
   1|   0|   1|   1|   0|   0|   1|   0|   1|
----+----+----+----+----+----+----+----+----+
   2|   1|   0|   1|   1|   0|   0|   0|   1|
----+----+----+----+----+----+----+----+----+
   3|   1|   1|   0|   1|   1|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   4|   0|   1|   1|   0|   1|   0|   1|   0|
----+----+----+----+----+----+----+----+----+
   5|   0|   0|   1|   1|   0|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   6|   1|   0|   1|   0|   1|   0|   1|   0|
----+----+----+----+----+----+----+----+----+
   7|   0|   0|   0|   1|   0|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   8|   1|   1|   0|   0|   0|   0|   0|   0|
----+----+----+----+----+----+----+----+----+

Nhap dinh bat dau duyet: 1
Thu tu duyet DFS tu dinh 1 la: 1 2 3 4 5 6 7 8
Press any key to continue . . .
```

```
D:\CAU TRUC ROI RAC\Bai2.e:

Nhap so luong dinh cua do thi: 8

--- NHAP MA TRAN KE (0 hoac 1) ---
Hang 1 (nhap 8 so): 0 1 1 0 0 1 0 1
Hang 2 (nhap 8 so): 1 0 1 1 0 0 0 1
Hang 3 (nhap 8 so): 1 1 0 1 1 1 0 0
Hang 4 (nhap 8 so): 0 1 1 0 1 0 1 0
Hang 5 (nhap 8 so): 0 0 1 1 0 1 0 0
Hang 6 (nhap 8 so): 1 0 1 0 1 0 1 0
Hang 7 (nhap 8 so): 0 0 0 1 0 1 0 0
Hang 8 (nhap 8 so): 1 1 0 0 0 0 0 0

 MA TRAN KE DA NHAP:
    |   1|   2|   3|   4|   5|   6|   7|   8|
----+----+----+----+----+----+----+----+----+
   1|   0|   1|   1|   0|   0|   1|   0|   1|
----+----+----+----+----+----+----+----+----+
   2|   1|   0|   1|   1|   0|   0|   0|   1|
----+----+----+----+----+----+----+----+----+
   3|   1|   1|   0|   1|   1|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   4|   0|   1|   1|   0|   1|   0|   1|   0|
----+----+----+----+----+----+----+----+----+
   5|   0|   0|   1|   1|   0|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   6|   1|   0|   1|   0|   1|   0|   1|   0|
----+----+----+----+----+----+----+----+----+
   7|   0|   0|   0|   1|   0|   1|   0|   0|
----+----+----+----+----+----+----+----+----+
   8|   1|   1|   0|   0|   0|   0|   0|   0|
----+----+----+----+----+----+----+----+----+

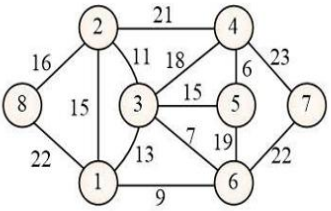
Nhap dinh bat dau duyet: 5
Thu tu duyet DFS tu dinh 5 la: 5 3 1 2 4 7 6 8
Press any key to continue . . .
```

2.8. Kết luận

- Chương trình đã thực hiện được các yêu cầu:

- Đọc đồ thị vô hướng, lưu dưới dạng ma trận kề.
- Nhận đỉnh bắt đầu từ người dùng và thực hiện thuật toán DFS đệ quy.
- Xuất ra dãy đỉnh được duyệt theo thứ tự DFS.
- Kiểm tra bằng tay (mô phỏng từng bước) khớp với kết quả chạy máy.

<u>Câu 3</u>: Viết chương trình đọc vào một đơn đồ thị vô hướng ở dạng danh sách kề, sau đó nhập vào hai đỉnh s và t, và áp dụng thuật toán Dijkstra để tìm đường đi ngắn nhất từ đỉnh s đến đỉnh t. Áp dụng cho đồ thị dưới đây, chạy code bằng computer và bằng tay, vẽ tay cây đường đi ngắn nhất tương ứng, chụp hình các kết quả trình bày vào file word.



3.1. Mục tiêu

- Viết chương trình nhập vào một đơn đồ thị vô hướng dưới dạng danh sách kề, áp dụng thuật toán Dijkstra vào đồ thị, sau đó xuất ra danh sách kề, nhập đỉnh s,t rồi áp dụng thuật toán để tìm đuòng đi ngắn nhất.

3.2. Cơ sở lý thuyết

- Dijkstra là thuật toán tìm đường đi ngắn nhất từ một đỉnh nguồn s đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm. Thuật toán sử dụng hàng đợi ưu tiên (priority queue) để luôn mở rộng đỉnh có khoảng cách nhỏ nhất trước.

- Danh sách kề (Adjacency List) là phương pháp biểu diễn đồ thị bằng cách, với mỗi đỉnh u, lưu trữ danh sách tất cả các đỉnh v kề với u. Nếu đồ thị có trọng số, mỗi phần tử trong danh sách lưu cả trọng số của cạnh tương ứng.

3.3. Thuật toán, Cấu trúc dữ liệu, Ưu và nhược điểm

- Cấu trúc dữ liệu:

- Danh sách kề (vector<pii> graph []): Lưu trữ các đỉnh và trọng số cạnh. Tối ưu bộ nhớ cho đồ thị thưa so với ma trận kề.
- Hàng đợi ưu tiên (priority_queue): Sử dụng cơ chế Min-Heap để luôn lấy ra đỉnh có khoảng cách ngắn nhất, tăng tốc độ tìm kiếm.
- Mảng bổ trợ:
  - dist []: Lưu khoảng cách ngắn nhất từ nguồn.
  - prev []: Lưu vết đỉnh đứng trước để truy xuất đường đi.
  - visited []: Đánh dấu đỉnh đã tối ưu xong để tránh lặp.

- Thuật toán:

- Nguyên lý: Thuật toán liên tục chọn đỉnh có nhãn khoảng cách nhỏ nhất chưa xét để cập nhật (relax) các đỉnh kề.

- Ưu và nhược điểm:

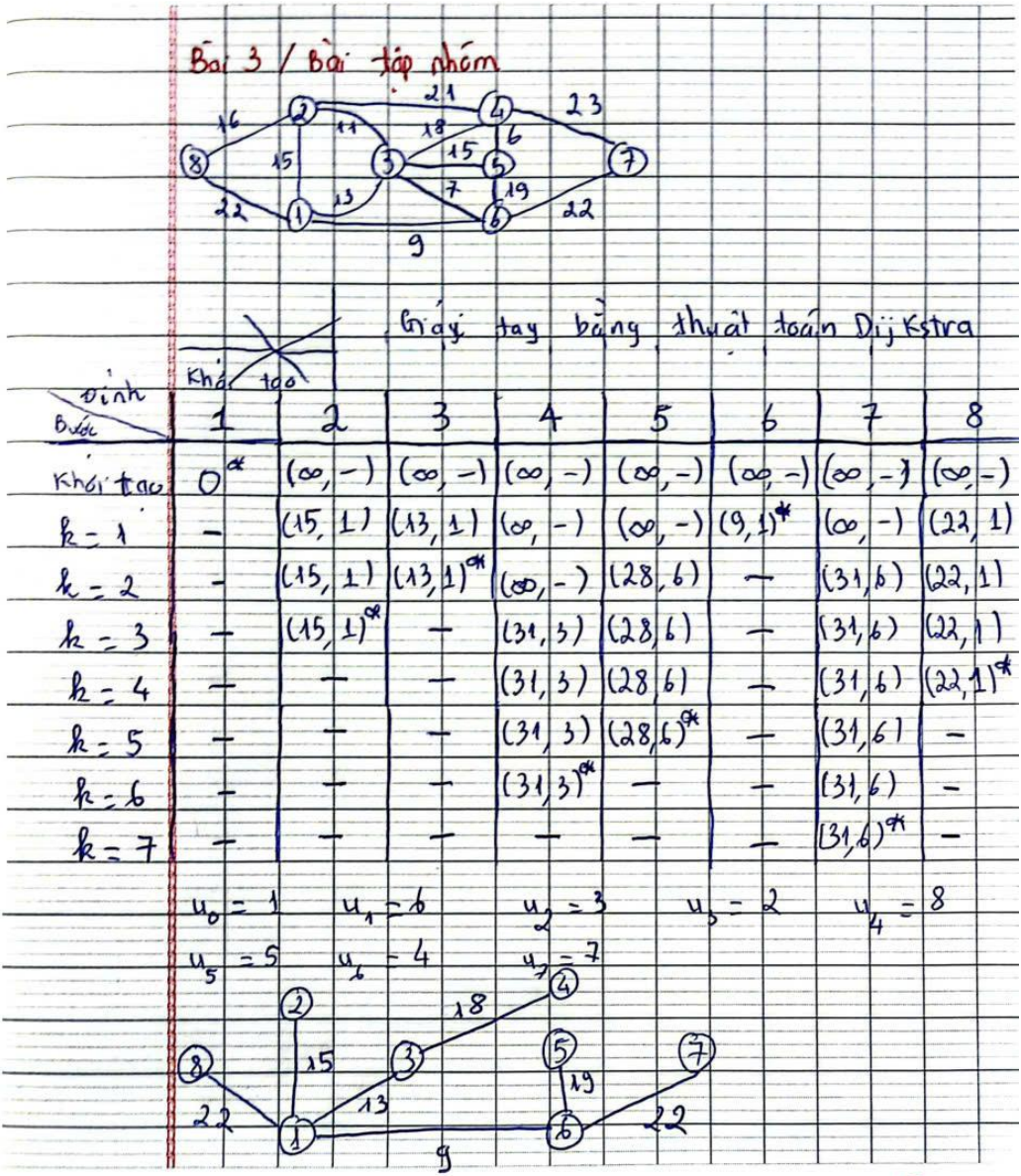
- **Ưu điểm:** Tốc độ nhanh, kết quả chính xác tuyệt đối với trọng số dương. Đảm bảo tìm được đường đi tối ưu nhất cho đồ thị có trọng số dương.
- **Nhược điểm:** Không hoạt động được với đồ thị có cạnh âm; lãng phí bộ nhớ hơn ma trận kề nếu đồ thị quá dày**.** Thuật toán sẽ cho kết quả sai nếu đồ thị có cạnh mang trọng số âm.

3.4. Danh sách kề của đồ thị

u	Ke(u)
1	2(15) 3(13) 6(9) 8(22)
2	1(15) 3(11) 4(21) 8(16)
3	1(13) 2(11) 4(18) 5(15) 6(7)
4	2(21) 3(18) 7(23) 5(6)

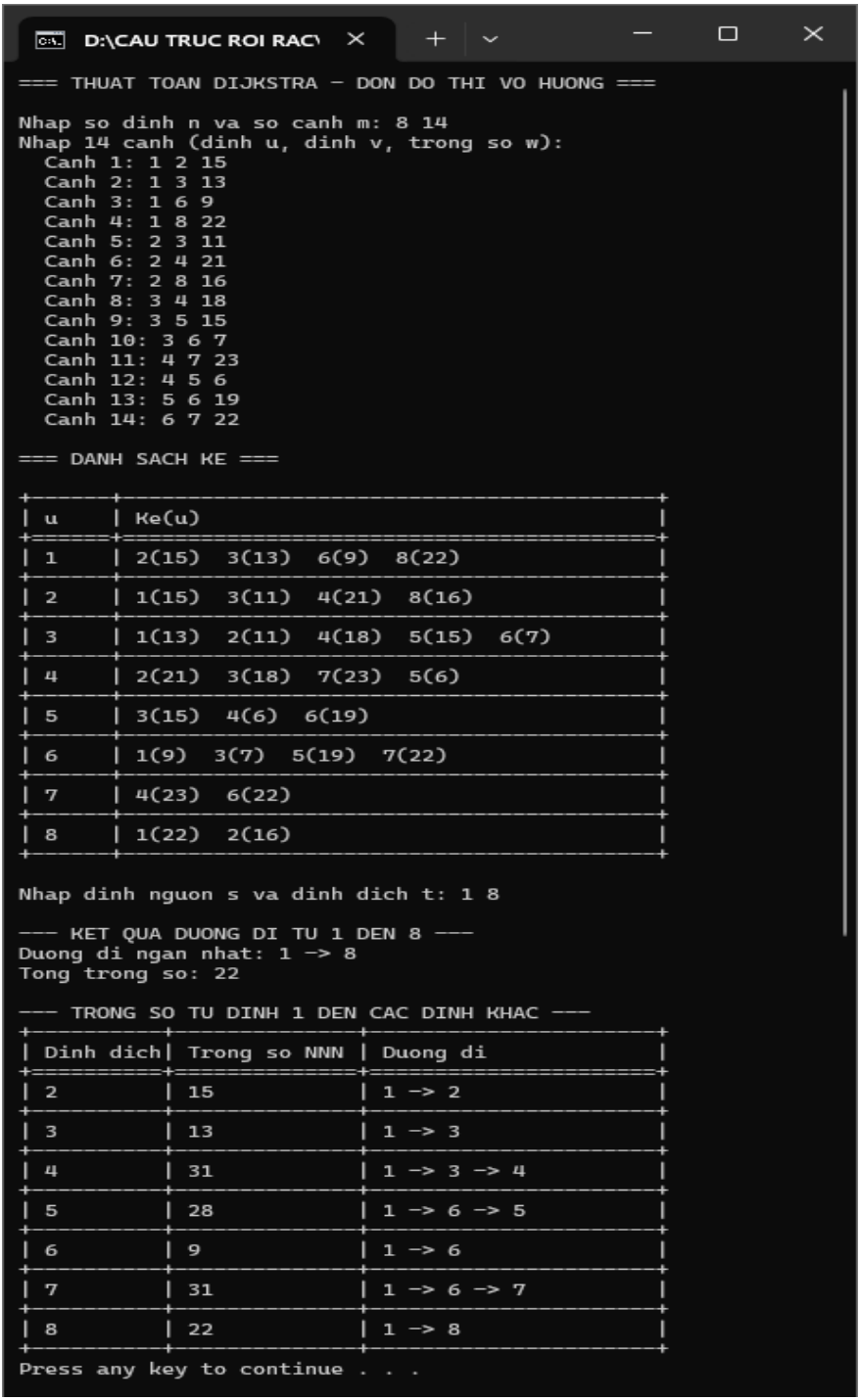
u	Ke(u)
5	3(15) 4(6) 6(19)
6	1(9) 3(7) 5(19) 7(22)
7	4(23) 6(22)
8	1(22) 2(16)

3.5. Giải bài toán bằng tay



TTBOOK
Được quét bằng CamScanner
Được quét bằng CamScanner

3.6. Giải bài toán bằng code



3.7. Kết luận

Qua bài trên, chúng ta đã thực hiện được các nội dung sau:

- Biểu diễn đồ thị vô hướng có trọng số bằng danh sách kề.
- Hiểu và cài đặt đúng thuật toán Dijkstra sử dụng hàng đợi ưu tiên.
- Chạy tay từng bước thuật toán để kiểm tra tính chính xác của chương trình.
- Vẽ cây đường đi ngắn nhất tương ứng với kết quả tìm được.